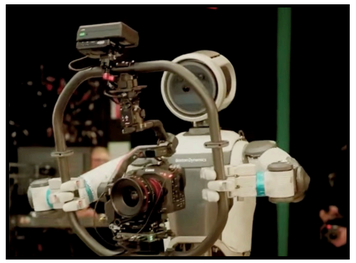
Follow-Ahead Human-Robot Navigation via Monte Carlo Tree Search and Deep Reinforcement Learning

Archita Srivastava, Ankush Singh, Gemmin Sugiura, Jonathan Ung
Simon Fraser University

Abstract

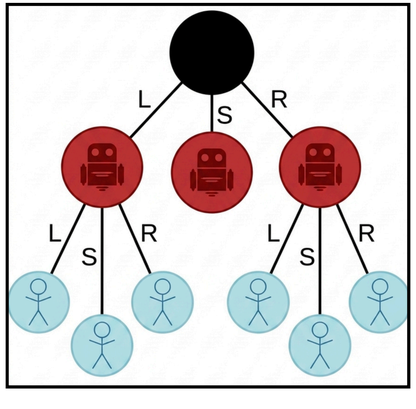
We replicate a methodology for robotic follow-ahead applications that addresses obstacle and occlusion avoidance. A high-level decision-making algorithm generates short-term navigational goals by integrating Monte Carlo Tree Search (MCTS) with Deep Reinforcement Learning (DRL). Experiments demonstrate that MCTS-DRL outperforms standalone MCTS and DRL baselines across obstacle-free and obstacle-present environments.



Applications

Follow-ahead robots navigate in front of a target person while maintaining a safe distance and avoiding obstacles in real time. Real-world use cases include:

- Autonomous shopping carts
- Filming robots that stay ahead of people
- Security robots that should stay in front of people



Methods

Pure DRL cannot generalize to unseen obstacle configurations since it would overfit to whatever environments it was trained on, making retraining necessary for every new setting. The solution is to train DRL only on the follow-ahead behavior in an obstacle-free setting, where the Q-function learns to estimate the expected return for each action given the current relative pose of the robot and person.

$$Q^*(o,a) = \mathbb{E}[r - \gamma \max_{a'} Q^*(o',a')]$$

The core insight is substituting this learned Q-function directly into MCTS as the node evaluator replacing random rollouts with informed value estimates. This is where the two methods fuse: MCTS gets consistent node evaluations from DRL, while obstacle and occlusion avoidance is handled separately by pruning any candidate pose that violates the occupancy map, no retraining required.

$$\mathrm{UCB} = \frac{w}{n^c} + c\sqrt{\frac{\log n^p}{n^c}}$$

At each time step the tree expands over a 3-second horizon, selecting nodes by UCB to balance exploration and exploitation. The leaf node with the highest score at c=0c=0 becomes the robot's next navigational goal, passed to the ROS navigation stack for execution.

Algorithm 1: MCTS-DRL Approach

```
Data: robot pose = (x_r, y_r, θ_r)^{t_0}
Data: (x_h, y_h, θ_h)^{t_0}, ..., (x_h, y_h, θ_h)^{t_3}
Data: occupancy map
Result: navigational goal point
parent node = (x_r^{t_0}, y_r^{t_0}, θ_r^{t_0}, x_h^{t_0}, y_h^{t_0}, θ_h^{t_0});
for num of expansion do
    while parent node is not fully expanded do
        child ← simulate using an action;
        if no collision then
            if no occlusion then
                R ← Q(o, a_i);
                child node state ← simulate using the
                  action;
            else
                value ← −1;
            parent value ← parent value + value;
        else
            delete the child node;
    parent node ← leaf node with highest UCB;
goal point ← leaf node with highest UCB (c = 0);
```

References

[1] Leisiazar et al., IROS 2023.
[2] Nikdel et al., ICRA 2021.
[3] van Hasselt et al., AAAI 2016.